

Package ‘BOLD.NODE’

June 17, 2026

Title Search and Explore BOLD Dataset Releases Efficiently

Version 1.0.0

Description Provides efficient tools for exploring and transforming the Barcode of Life Data Systems (BOLD) data packages on local machines. It enables fast local querying of the data packages without relying on live API calls. Results can be easily converted into formats compatible with widely used R packages and third party tools.

License CC BY 4.0

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports data.table,
DBI,
dbplyr,
dplyr,
duckdb,
progressr,
rlang,
sf,
tidyr

Suggests Biostrings,
BiocManager

Contents

bcdm_field_names	2
bcdm_field_values	2
bcdm_to_dnastringset	3
bcdm_to_dwc	4
bcdm_to_fasta	5
bcdm_to_occmatrix	6
bcdm_to_sf	8
bold_parquet_search	9
bold_search_collect	11
get_bin_consensus	12
get_concise_summary	14

Index**16**

bcdm_field_names	<i>Retrieve metadata of the BOLD BCDM data fields</i>
------------------	---

Description

Provides information on the field (column) names and their respective data type, all of which are compliant with the Barcode Core Data Model (BCDM).

Usage

```
bcdm_field_names(print.output = FALSE)
```

Arguments

`print.output` Whether the output should be printed in the console. Default is FALSE.

Details

The function downloads the latest field (column) metadata (file type and brief description) for the Barcode Core Data Model (BCDM) from https://github.com/boldsystems-central/BCDM/blob/main/field_definitions.tsv; `output = TRUE` will print the information in the console. *Important Note:* Two field names 'country/ocean' and 'province/state' have been modified to 'country.ocean' and 'province.state' to match with BOLDconnectR output and for operational ease.

Value

A data frame containing definitions for all fields (columns).

Examples

```
bold.field.data <- bcdm_field_names()

head(bold.field.data, 10)
```

bcdm_field_values	<i>Extract unique values of different BCDM fields from the BOLD data package</i>
-------------------	--

Description

Extracts distinct values of specified field/s in the BOLD parquet data.

Usage

```
bcdm_field_values(
  input.data,
  specific.cols,
  save.data = FALSE,
  output.file = NULL
)
```

Arguments

<code>input.data</code>	Path to the input parquet file or the <code>bold_parquet_search</code> result.
<code>specific.cols</code>	Name of the column to extract unique values from.
<code>save.data</code>	Logical value indicating whether to save the results to disk as a .rds file (default: FALSE).
<code>output.file</code>	Path (without extension) for saving results as .rds file (required if <code>save.data = TRUE</code>).

Details

This function extracts unique values from one or more specified columns in BOLD parquet data. It handles both the parquet file and `tbl_sql` objects from `bold_parquet_search` as input. The results can be saved to disk as an .rds file for later use.

Value

A list containing unique values from the specified column. if `save.data = T`, a .rds file is exported locally.

Examples

```
## Not run:

# Search the BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Diptera",
  geography = "India",
  marker = "COI-5P"
)

# Get the field values

vocab.data <- bcdm_field_values(bold_search, bold_search,
  specific.cols = c("inst", "identified.by"))

## End(Not run)
```

`bcdm_to_dnastringset` *Convert the BOLD parquet search into a DNASTringSet object*

Description

Converts the sequence data from the search result into a `DNASTringSet` object for downstream multiple sequence alignment with customized headers.

Usage

```
bcdm_to_dnastringset(bold.search.res, marker = NULL, cols_for_seq_names)
```

Arguments

`bold.search.res`
A `tbl_sql` object containing BOLD search results.

`marker`
Character vector specifying the genetic marker.

`cols_for_seq_names`
Character vector of field names to include in the header.

Details

This function transforms the search results from `bold_parquet_search` into a `DNAStrngSet` object suitable for downstream data analyses. The `cols_for_seq_names` argument lets users create custom headers using the BCDM column names.

Value

A `DNAStrngSet` object.

Examples

```
## Not run:

# Search the BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Coleoptera",
  geography = "Canada",
  basecount = c(500, 660)
)

# Get the DNAStrngset object

bold.dnasrtingset <- bcdm_to_dnasrtingset(bold_search,
  marker = "COI-5P",
  cols_for_seq_names = c("processid", "family")
)

## End(Not run)
```

bcdm_to_dwc

Convert the BOLD parquet search in to a Darwin Core (DwC) format data model

Description

Converts `bold_parquet_search` results from BCDM format to Darwin Core Standard format.

Usage

```
bcdm_to_dwc(bold.search.res)
```

Arguments`bold.search.res`A `tbl_sql` object containing BOLD search results.**Details**

This function maps BCDM (Barcode Core Data Model) fields to their Darwin Core equivalents.

Important Note: All fields should be available in the `bold_parquet_search` `tbl_sql` object otherwise, the function will throw an error.

Value

A data frame with columns mapped to Darwin Core equivalent fields.

Examples

```
## Not run:

# Search the BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = c("Cerambycidae", "Meloidae"),
  geography = "Australia"
)

# Get the DNASTringset object

bold.dwc <- bcdm_to_dwc(bold_search)

## End(Not run)
```

bcdm_to_fasta

*Export the BOLD parquet search to FASTA format***Description**

Exports nucleotide sequences from BOLD search results to a FASTA file with customizable headers.

Usage

```
bcdm_to_fasta(bold.search.res, output.file, fas.header, chunk.size = 1e+06)
```

Arguments`bold.search.res`A `tbl_sql` object containing BOLD search results.`output.file`

Path to the output FASTA file.

`fas.header`

Character vector of field names to include in the FASTA header.

`chunk.size`

Number of records to process in each chunk (default: 1000000).

Details

This function transforms the search results from `bold_parquet_search` into a FASTA file. A data chunking option is available to manage large sizes to avoid memory issues. The `fas.header` argument lets users create custom headers using the BCDM field names (Metadata on fields can be checked using the `bcdm_field_names` function).

Value

writes a FASTA file to disk with custom headers as specified by the user.

Examples

```
## Not run:

# Search the BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Coleoptera",
  geography = "Canada",
  marker = "COI-5P",
  basecount = c(500, 660)
)

# Get a fasta file
bcdm_to_fasta(
  bold_search,
  output.file = "trial2.fas",
  fas.header = c("bin_uri", "processid")
)

## End(Not run)
```

bcdm_to_occmatrix

Convert the BOLD parquet search into an occurrence matrix

Description

Extracts occurrence data (specimen counts by taxon and location) from BOLD search results.

Usage

```
bcdm_to_occmatrix(
  bold.search.res,
  kingdom = "Animalia",
  taxon.rank,
  taxon.name = NULL,
  site.cat = NULL,
  pre.abs = FALSE
)
```

Arguments

<code>bold.search.res</code>	A <code>tbl_sql</code> object containing BOLD search results.
<code>kingdom</code>	Character value specifying the kingdom (default: Animalia).
<code>taxon.rank</code>	Taxonomic rank to aggregate by (kingdom, phylum, class, order, family, genus, or species).
<code>taxon.name</code>	Optional vector of specific taxon names to include.
<code>site.cat</code>	Optional categorical variable to group occurrence data by (e.g., region).
<code>pre.abs</code>	Logical indicating whether to convert counts to presence/absence (1/0) data (default: FALSE).

Details

This function transforms the search results from `bold_parquet_search` into occurrence data matrices used commonly in biodiversity and ecological analyses by packages like `vegan` and `betapart`. The `kingdom` argument specifies the taxa kingdom with the default value being `Animalia`. The occurrences differ based on the kingdom. For `Animalia`, only records having BINs are counted (i.e., records without BINs are removed before calculations), while for other kingdoms, all records having a sequence are counted (i.e., records without sequences are removed before calculations). It supports aggregation at different taxonomic ranks (from kingdom to species) for a single or multiple taxa (also applicable for `bin_uri`; the difference being that the numbers in each cell would be the number of times that respective BIN is found at a particular `site.cat`), with optional filtering by specific taxon names. `site.cat` can be any of the geography fields (Metadata on fields can be checked using the `bcdm_field_names` function). In case where `site.cat = NULL`, the column `coord` will be used as default. The function can convert count data to presence/absence (1/0) format. *Important Note:* The `bcdm_to_occmatrix` function requires taxonomy (including `bin_uri`) and geography fields to be available in the `bold_parquet_search` result. In case the `specific.cols` argument is used in the `bold_parquet_search` function to retrieve certain columns that don't have taxonomy and geography columns, the function will throw an error.

Value

A data frame with occurrence data (taxon names as columns, site categories or coordinates as rows).

Examples

```
## Not run:

# Search the BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Odonata",
  geography = "Thailand"
)

# Get the occurrence matrix

occurrence_data <- bcdm_to_occmatrix(
  bold_search,
  kingdom = "Animalia",
  taxon.rank = "family",
  site.cat = "region"
```

```
)
## End(Not run)
```

bcdm_to_sf

Convert the BOLD parquet search to spatial point dataframe

Description

Converts BOLD search results with coordinate data to spatial points (sf object).

Usage

```
bcdm_to_sf(bold.search.res, chunk.size = 1e+05)
```

Arguments

<code>bold.search.res</code>	A <code>tbl_sql</code> object containing BOLD search results.
<code>chunk.size</code>	Number of records to process in each chunk (default: 100000).

Details

This function transforms the search results from `bold_parquet_search` into an `sf` object. A data chunking option is available to manage large sizes to avoid memory issues. The function creates point geometries in the WGS84 coordinate system (EPSG:4326). Records that don't have coordinate data are removed during processing.

Value

An `sf` object with point geometry in WGS84 coordinate system (EPSG:4326).

Examples

```
## Not run:

# Search the BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Odonata",
  geography = "Malaysia"
)

# Get the occurrence matrix

sf_data <- bcdm_to_sf(bold_search, chunk.size = 100000)

## End(Not run)
```

bold_parquet_search	<i>Search the BOLD public data packages available in parquet format</i>
---------------------	---

Description

Search the BOLD public data packages available in the parquet format based on various search criteria including taxonomy, geography, institutes etc.

Usage

```
bold_parquet_search(
  input.parquet,
  ids = NULL,
  bins = NULL,
  taxonomy = NULL,
  geography = NULL,
  institutes = NULL,
  identified.by = NULL,
  seq.source = NULL,
  marker = NULL,
  basecount = NULL,
  biogeo.cat = NULL,
  dataset.projects = NULL,
  bounding.box = NULL,
  ambi.base.cutoff = NULL,
  specific.cols = NULL
)
```

Arguments

input.parquet	Path to the input parquet file.
ids	Vector of process IDs or sample IDs to filter by.
bins	Vector of BIN numbers (i.e., URIs) to filter by.
taxonomy	Vector of taxonomic names to filter by (can include kingdom, phylum, class, order, family, subfamily, genus, species).
geography	Vector of geographic locations to filter by (can include country.ocean, province.state, region, sector, site).
institutes	Vector of institute codes to filter by.
identified.by	Vector of identifiers to filter by.
seq.source	Vector of sequence run sites to filter by.
marker	Vector of marker codes to filter by.
basecount	Nucleotide base count filter - either a single value or a vector of two values for a range.
biogeo.cat	Vector of biogeographic/ecological categories to filter by (biome, realm, or ecoregion).
dataset.projects	Vector of dataset/project codes to filter by.

`bounding.box` Numeric vector of length 4: `c(min_lon, max_lon, min_lat, max_lat)`.

`ambi.base.cutoff` Character value for filtering data based proportion of ambiguous bases (IUPAC codes). Three values currently available ("`<1%`", "`1-5%`", or "`>5%`"). Default value is `NULL`.

`specific.cols` Optional character vector of specific columns to return.

Details

This function loads the BOLD public data packages parquet files (<https://boldsystems.org/data/data-packages/>) via DuckDB and applies filters based on the provided parameters. It supports filtering by ids (sampleid, processid), taxonomy (from kingdom to species level), geography (country to site level), biogeography (biome to ecoregion level), BINs, institutes, identifiers, sequence sources, genetic markers, nucleotide base counts, dataset or projects, spatial bounding boxes, and ambiguous base percent cutoffs. Users can also specify particular columns to return using the `specific.cols` parameter (column names can be checked using the `bcdm_field_names` function). The `tbl_sql` object can then be used by any of the `bcdm_to_*` functions for data transformations or `bold_search_collect` to load all the data in memory.

Value

A `tbl_sql` object containing the filtered data. Total records in the search are printed on the console.

Examples

```
## Not run:

# Search the BOLD data package

# Taxonomy
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = c("Odonata", "Poecilia")
)

# Geography
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  geography = "Canada"
)

# Combination of many search criteria
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Coleoptera",
  geography = "Canada",
  marker = "COI-5P",
  basecount = c(500, 660)
)

## End(Not run)
```

bold_search_collect *Collect and export parquet search results*

Description

collects, outputs and exports the tbl_sql bold_parquet_search results object, processing large datasets in user defined manageable chunks.

Usage

```
bold_search_collect(
  bold.search.res,
  chunk.size = 1e+06,
  sys.sleep = 0,
  export = FALSE,
  export.type = c("tsv", "parquet"),
  output.path = NULL
)
```

Arguments

bold.search.res	A tbl_sql object obtained from bold_parquet_search.
chunk.size	Maximum number of rows to process in each chunk. Default value is 1e6.
sys.sleep	Time to sleep between chunks in seconds. Default value is 0.
export	Logical value that allows user to export the output locally. Default value is FALSE.
export.type	Character string specifying the data type of the exported file (tsv or parquet).
output.path	Character string specifying the local path for data export along with the file name and extension.

Details

This function collects (loads it in the R session) the search results from bold_parquet_search. Data chunking and system sleep options are available to manage large sizes to avoid memory issues. The function also supports exporting the searches in either TSV or Parquet format. export=FALSE (default) returns the collected data in the R session only. The full output_path has to be provided (along with the intended file name and file extension) when export=TRUE. *Important Note:* Some data searches (e.g., all Diptera) can get very large and overload the RAM capacity on some machines.

Value

A data frame containing all collected results; if export = TRUE, either a TSV or parquet file exported locally.

Examples

```
## Not run:

# Search the BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Coleoptera",
  geography = "Canada",
  marker = "COI-5P",
  basecount = c(500, 660)
)

# Collect the data (no export)
bold_search_collect(
  bold_search,
  chunk.size = 50000,
  export = FALSE
)

# Collect and export
bold_search_collect(
  bold_search,
  chunk.size = 50000,
  export = TRUE,
  export.type = "parquet",
  output.path = "userdefinedpath"
)

## End(Not run)
```

get_bin_consensus

Compute consensus BIN taxonomy for BOLD data package

Description

Computes and returns consensus taxonomic identifications for each BIN in search results or a BCDM data frame.

Usage

```
get_bin_consensus(
  bold.search.res = NULL,
  bold.df = NULL,
  ranks = c("kingdom", "phylum", "class", "order", "family", "subfamily", "tribe",
    "genus", "species", "subspecies"),
  threshold = 1,
  min.ids = 1,
  enforce.scientific = TRUE,
  groups = "bin_uri",
  discord.format = c("text", "list")
)
```

Arguments

<code>bold.search.res</code>	A <code>tbl_sql</code> object obtained from bold_parquet_search . Optional: one of <code>bold.search.res</code> or <code>bold.df</code> must be provided.
<code>bold.df</code>	Data frame in BCDM format, or any <code>data.frame</code> or <code>data.table</code> minimally containing <code>bin_uri</code> (or other grouping variable) and taxonomic identifications for all available records. Optional: one of <code>bold.search.res</code> or <code>bold.df</code> must be provided.
<code>ranks</code>	A character vector of ranks to consider for consensus identifications. Defaults to the standard BOLD ranks.
<code>threshold</code>	Numeric value(s) between 0 and 1 indicating the minimum proportion of records in a BIN that must have a concordant identification in order to establish a consensus. Supply as a single value, a vector of length equal to the number of ranks in consideration, or a named list with names corresponding to ranks. If supplied as a named list, an optional "default" value can be set for any ranks that are not explicitly specified (e.g., <code>threshold = list(species = 0.95, default = 0.75)</code>). Default value is 1.0 (i.e., strict consensus at all ranks).
<code>min.ids</code>	Numeric value(s) indicating the minimum number of identifications needed to establish a consensus (names with fewer identifications are still included when calculating proportions). Supply as a single value, a vector of length equal to the number of ranks in consideration, or a named list with names corresponding to ranks. If supplied as a named list, an optional "default" value can be set for any ranks that are not explicitly specified (e.g., <code>min.ids = list(family = 1, default = 2)</code>). Default value is 2 (i.e., min 2 identifications at any rank).
<code>enforce.scientific</code>	A logical value indicating whether non-scientific, provisional names should be ignored when determining consensus. Default value is TRUE, meaning non-scientific names are ignored.
<code>groups</code>	Grouping variable. Default value is "bin_uri".
<code>discord.format</code>	String indicating the desired output format for the <code>discordant_ids</code> column. Can be one of "text", or "list". If "text" (the default), the output is a string column with comma-separated values in the format "Taxon (proportion)". If "list", the output is a list column with names indicating competing identifications and values indicating proportions of discordant identifications for each taxon.

Details

Consensus is defined as any name that exceeds the specified threshold, expressed as a proportion of records with a concordant identification (i.e., same name, same rank). The function steps backwards through the eligible ranks to determine the lowest available concordant identification that meets the criteria specified by `threshold`, `min.ids`, and `enforce.scientific`. Different thresholds can be supplied for each rank, if desired (either as a vector of equal length to ranks or as a named list). The function can also be applied to any other grouping variable by modifying `groups`.

Important Note: As this function performs operations on the input data, it may be quite slow for very large data sets and/or weaker machines. Please check the size of `bold.search.res` input objects using [get_concise_summary](#) and proceed with caution.

Value

A table of consensus identifications for each BIN (or other grouping variable), with the following columns: `bin_uri`, `member_count`, `concordant_rank`, `concordant_id`, `discordant_rank`,

```
discordant_ids.
```

Examples

```
## Not run:

# Search for BOLD data package
bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Coleoptera",
  geography = "Canada"
)

# Compute strict consensus identifications for BINs in searched data
strict_consensus <- get_bin_consensus(
  bold.search.res = bold_search,
  threshold = 1.0
)

# Compute identifications concordant among at least 75% of BIN members,
# as long as at least three records carry the majority identifications
# in each BIN.
bin_ids <- get_bin_consensus(
  bold.search.res = bold_search,
  threshold = 0.75,
  min.ids = 3
)

# Include non-scientific names (i.e., interim taxonomy or placeholder names)
# in consideration of BIN consensus.
bin_consensus <- get_bin_consensus(
  bold.search.res = bold_search,
  threshold = 0.9,
  enforce.scientific = FALSE
)

## End(Not run)
```

get_concise_summary	<i>Generate a concise summary of the search results</i>
---------------------	---

Description

Creates a summary statistics table from the `bold_parquet_search` `tbl_sql` object.

Usage

```
get_concise_summary(bold.search.res)
```

Arguments

`bold.search.res`

A `tbl_sql` object containing `bold_parquet_search` results.

Details

The function provides a concise summary of the search obtained by `bold_parquet_search` that includes details like total records, unique BINs, unique institutes, unique markers and amplicon size range.

Value

A data frame with the summary statistics.

Examples

```
## Not run:

# Search the BOLD data package

parquet_file <- "user defined path to parquet file"

bold_search <- bold_parquet_search(
  input.parquet = parquet_file,
  taxonomy = "Hemiptera",
  geography = "India",
  marker = "COI-5P"
)
# Get the concise summary
bold_summary <- get_concise_summary(bold_search)

## End(Not run)
```

Index

bcdm_field_names, [2](#)
bcdm_field_values, [2](#)
bcdm_to_dnastringset, [3](#)
bcdm_to_dwc, [4](#)
bcdm_to_fasta, [5](#)
bcdm_to_occmatrix, [6](#)
bcdm_to_sf, [8](#)
bold_parquet_search, [9](#), [13](#)
bold_search_collect, [11](#)

get_bin_consensus, [12](#)
get_concise_summary, [13](#), [14](#)